MyCodeSchool

React — apeluri la backend

fișă rapidă · fetch, useEffect, erori, POST / PATCH / DELETE, token

1. Drumul unei cereri

```
browser :5173 ── fetch("/users-api/api/users") ──► Vite (proxy) ──► localhost:8111/api/users ──► users-api ──► MySQL
              ◄── response: status 200 + corp JSON ◄──────────────────────────────────────────┘

/users-api ──► :8111  (fără login)      /jobs-api   ──► :8107  (fără login)
/shop-api  ──► :8104  (cu login)        /school-api ──► :8105  (cu login)
```

2. Un GET, pas cu pas

```
async function load(): Promise<void> {
  const response = await fetch(url);          1. trimite, așteaptă
  const data: User[] = await response.json(); 2. corpul → obiecte
  setUsers(data);                             3. redesenează
}
```

```
la click     <button onClick={load}>
la pornire   useEffect(() => { load(); }, []);
                          [] = o singură dată
dev: apare de 2 ori ──► StrictMode, normal
```

3. Răspunsul: ok, eroare, server oprit

```
try {
  const response = await fetch(url);
  if (!response.ok) {              4xx / 5xx: fetch NU aruncă
    const error = await response.json();
    setError(`${response.status}: ${error.message}`);
    return;
  }
  setUsers(await response.json());
} catch {                          serverul nu răspunde deloc
  setError("Serverul nu răspunde");
} finally {
  setLoading(false);               rulează mereu
}
```

200	OK · 201 creat
400	date greșite · details: ["câmp: mesaj"]
401	fără token / token greșit
404	nu există · message
409	conflict: email deja folosit

```
loading = true  ──► „Se încarcă...”
          false ──► lista sau eroarea
```

4. Trimiți date

```
await fetch("/users-api/api/users", {
  method: "POST",                  PATCH · PUT · DELETE
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ firstName, email, ... }),
});                                DELETE: fără body
```

```
parametri în URL  `...?age=${age}&size=2`
POST   ──► user nou       setUsers([...users, user])
PATCH  ──► user schimbat  users.map(u => u.id === x.id ? x : u)
DELETE ──► dacă ok        users.filter(u => u.id !== id)
```

5. Job queue: răspunsul vine mai târziu

```
POST /jobs ──► { jobId }      GET /jobs/{jobId} ──► { status }    0 În coadă · 1 Rulează · 2 Gata · 3 Eșuat
                               întrebi din nou:  while (status < 2) { await wait(500); ... }
```

6. Login + token

```
POST /api/auth/login { email, password } ──► { accessToken } ──► setToken(...)
GET  /api/products  headers: { Authorization: `Bearer ${token}` } ──► 200       fără token ──► 401
logout = setToken("")
```

7. Backend-urile

users-api	http://localhost:8111/swagger-ui.html	/api/users + /older-than, /average-age, /search... · modificare = PATCH
job-queue-api	http://localhost:8107/swagger	POST /jobs, GET /jobs, GET /jobs/{id}
shop-api	http://localhost:8104/swagger-ui.html	login admin@shop.com / admin1234 · /api/products
school-api	http://localhost:8105/swagger-ui.html	login admin@online-school.ro / password · /courses